🤖 NEW PROJECT — ZERO TO SPEC WITH IBM BOB

Starting a New Project from Scratch

Everything you need to create a new project, configure the spec-driven workflow, and write your first feature spec before touching any code — using IBM Bob.

🤖 Tool: IBM Bob | ⏱️ Setup Time: ~20 minutes | 📦 Requires: universal-spec-mcp.zip

1 Create Project → 2 Add MCP Server → 3 Fill Steering → 4 Open in Bob → 5 Write First Spec → 6 Start Coding

WHAT YOU WILL BUILD

By the end of this guide you will have a new project where every feature starts with a spec — not code. IBM Bob will automatically produce three files before writing a single line of implementation:

📋 **Requirements** `requirements.md` → 🏗️ **Design** `design.md` → ✅ **Tasks** `tasks.md` → 💻 **Code** `implementation`

1 Create Your New Project Folder

Set up the base directory structure for your project

Create the Project Folder

Create a new folder anywhere on your machine. This will be your project root. For example:

```
Terminal

$ mkdir my-new-project
$ cd my-new-project
```

On **Windows**, simply right-click on your Desktop or in File Explorer and choose **New → Folder**. Name it `my-new-project`.

Open the Folder in IBM Bob

Open IBM Bob, then go to **File → Open Folder** and select your new project folder. Bob will open it as your workspace. Alternatively, from the terminal:

```
Terminal

$ bob .
Opening my-new-project in IBM Bob...
```

> 💡 You can also drag the folder directly onto the IBM Bob icon to open it instantly.

Initialise Your Project (Optional but Recommended)

If you are building a Python backend, initialise it now. For a Node.js project, run `npm init -y`. For Python:

```
Terminal — IBM Bob Integrated Terminal (Ctrl+`)

$ python -m venv .venv
$ .venv\Scripts\activate  # Windows
$ pip install fastapi sqlalchemy alembic
Successfully installed fastapi sqlalchemy alembic
```

> 💜 The spec-driven workflow works with **any tech stack**. The MCP server generates plain Markdown files — it does not depend on your language or framework.

2 Add the MCP Server to Your Project

Copy the server files and connect them to IBM Bob

Extract the MCP Server

Extract `universal-spec-mcp.zip` to a permanent location on your machine — **not inside your project folder**. The server is shared across all your projects.

```
📁 C:\Tools\universal-spec-mcp\
```

> ⚠️ Do not put the MCP server inside your project folder. It is a shared tool, like a compiler — it lives separately and serves all your projects.

Install the Python Dependencies

Open a terminal, navigate to the extracted server folder, and install the required packages:

```
Terminal

$ cd C:\Tools\universal-spec-mcp
$ pip install fastmcp pydantic
Successfully installed fastmcp pydantic
```

Copy the `.bob/` Config Folder into Your Project

Inside the extracted zip, find the `.bob/` folder. Copy it into your new project root. This gives your project the steering templates and rules.

```
Project structure after copying

my-new-project/
├── .bob/                  ← copy this from the zip
│   ├── mcp.json           ← registers the MCP server
│   ├── modes/
│   │   └── spec-architect.json
│   ├── rules/
│   │   └── spec-workflow.md
│   └── steering/
│       ├── product.md     ← you will fill this in Step 3
│       ├── tech.md        ← you will fill this in Step 3
│       └── structure.md   ← you will fill this in Step 3
└── src/                   ← your project code goes here
```

Update the Path in `.bob/mcp.json`

Open `.bob/mcp.json` in your project and update the path to match where you extracted the server:

.bob/mcp.json

```
{
  "mcpServers": {
    "universal-spec-architect": {
      "command": "python",
      "args": [
        "C:/Tools/universal-spec-mcp/src/universal_spec_mcp/server.py"
      ],
      "env": {
        "PYTHONPATH": "C:/Tools/universal-spec-mcp/src"
      }
    }
  }
}
```

ℹ️ Replace `C:/Tools/universal-spec-mcp` with the actual path where you extracted the zip. Use forward slashes even on Windows.

3 Fill In Your Steering Files

Tell Bob about your product, tech stack, and project structure — this is its permanent memory

What Are Steering Files?

Steering files are Bob's **persistent memory** about your project. Without them, Bob makes generic decisions. With them, it knows your exact tech stack, naming conventions, and business goals — and every spec it generates will fit your project perfectly from day one.

> 💡 Think of steering files as the briefing document you would give a new engineer joining your team. Fill them in once and Bob reads them automatically on every request.

Fill in `product.md` — What Your Product Does

Open `.bob/steering/product.md` and fill in the template. Here is an example for a new expense management app:

product.md

PRODUCT NAME

SmartExpense — AI-Powered Expense Management

ONE-LINE DESCRIPTION

A web application that automates expense report submission, approval, and payroll integration for enterprise teams.

PRIMARY USERS

Employees (submit expenses), Managers (approve/reject), Finance team (audit and reporting)

CORE FEATURES

Receipt upload, expense submission, manager approval workflow, duplicate detection, payroll integration

BUSINESS OBJECTIVES

Reduce expense processing time by 80%. All PII must be encrypted at rest. Reports must be processed within 24 hours.

> 💜 The **Business Objectives** field is the most important. Constraints like "PII must be encrypted" and "24-hour SLA" will automatically appear in every spec Bob generates.

Fill in `tech.md` — Your Technology Stack

Open `.bob/steering/tech.md`. This prevents Bob from suggesting the wrong framework or library.

tech.md

BACKEND

Python 3.11, FastAPI 0.111, SQLAlchemy 2.0, Alembic (migrations)

FRONTEND

React 18, TypeScript, IBM Carbon Design System

DATABASE

PostgreSQL 15 (production), SQLite (local dev)

FILE STORAGE

IBM Cloud Object Storage — max 10MB per file, PDF/PNG/JPG only

HARD CONSTRAINTS

No raw SQL — use SQLAlchemy ORM only. All API responses must use the standard envelope: {data, error, meta}.

Fill in `structure.md` — File Organisation

Open `.bob/steering/structure.md`. This ensures every file Bob creates lands in the right place with the right name.

structure.md

DIRECTORY TREE

backend/src/api/ (routes), backend/src/services/ (business logic), backend/src/models/ (DB models), frontend/src/components/, frontend/src/pages/

NAMING CONVENTIONS

Python files: snake_case. React components: PascalCase. DB tables: plural snake_case. API routes: /api/v1/{resource}

IMPORT STYLE

Absolute imports only. No relative imports. Use src/ as root.

4 Verify Bob Recognises the MCP Server

Confirm all 6 tools are connected and switch to Spec Architect mode

🔍

Check the MCP Tools Panel

In IBM Bob, open the **MCP Tools panel** (look for the plug icon in the sidebar or go to **Settings → MCP Servers**). You should see `universal-spec-architect` listed with 6 tools all showing as **Connected**:

MCP TOOLS — UNIVERSAL-SPEC-ARCHITECT	
✅ Connected	initialize_spec
✅ Connected	write_requirements
✅ Connected	write_design
✅ Connected	write_tasks
✅ Connected	update_task_status
✅ Connected	run_hook

⚠️ If any tool shows as **Error**, check that the path in `.bob/mcp.json` is correct and that `fast mcp` is installed. Run `python C:/Tools/universal-spec-mcp/src/universal_spec_mcp/server.py` in a terminal to see the error directly.

🎯

Switch to Spec Architect Mode

In the Bob chat panel, click the **mode selector** (usually shown as the current mode name at the top of the chat). Select **Spec Architect** from the list.

✅ When Spec Architect mode is active, Bob will **never write code first**. It will always call the MCP tools to produce requirements, design, and tasks before any implementation begins.

5 Write Your First Spec-Driven Feature

Type one message and watch the full Requirements → Design → Tasks cycle run automatically

Send Your First Message to Bob

In the Bob chat panel (Spec Architect mode active), type the following message and press Enter:

You

> I need to build a user authentication feature. Employees should be able to register with their IBM email, log in, and reset their password via email link.

Bob Calls the MCP Tools Automatically

Bob will immediately call the MCP tools in sequence — you will see each call appear in the chat as it happens:

MCP TOOL CALL 1 OF 4

initialize_spec

```
feature_name: "user-authentication"
workflow: "requirements-first"
```

MCP TOOL CALL 2 OF 4

write_requirements

```
feature_name: "user-authentication"
requirements: [
  "WHEN an employee submits a valid IBM email and password, THE SYSTEM SHALL create a new account and send a verification email within 30 seconds.",
  "WHEN an employee submits an invalid email format, THE SYSTEM SHALL display an inline error message without submitting the form.",
  "WHEN an authenticated employee requests a password reset, THE SYSTEM SHALL send a one-time link valid for 15 minutes.",
  "WHILE a password reset link is expired, THE SYSTEM SHALL redirect the employee to request a new link.",
  "WHEN an employee fails login 5 times consecutively, THE SYSTEM SHALL lock the account for 30 minutes and notify the employee by email."
]
```

MCP TOOL CALL 3 OF 4

write_design

```
feature_name: "user-authentication"
architecture: "JWT-based auth with FastAPI, IBM Cloud Object Storage for avatars..."
data_models: "User(id, email, hashed_password, is_verified, failed_attempts, locked_until)"
error_handling: "401 Unauthorized, 422 Validation Error, 429 Too Many Requests"
```

MCP TOOL CALL 4 OF 4

write_tasks

```
feature_name: "user-authentication"
tasks: [
  "TASK-001: Create User DB model and Alembic migration",
  "TASK-002: Implement /api/v1/auth/register endpoint",
  "TASK-003: Implement email verification service",
  "TASK-004: Implement /api/v1/auth/login with JWT",
  "TASK-005: Implement account lockout logic",
  "TASK-006: Implement /api/v1/auth/reset-password flow",
  "TASK-007: Write integration tests for all auth endpoints"
]
```

Review the Generated Spec Files

After the 4 tool calls complete, open your project in Bob's file explorer. You will see a new `.specs/` folder with three files:

```
my-new-project/.specs/user-authentication/requirements.md
```

```
my-new-project/.specs/user-authentication/design.md
```

```
my-new-project/.specs/user-authentication/tasks.md
```

> Review each file before proceeding. You can ask Bob to revise any requirement or design decision — just say "Change REQ-003 so the reset link is valid for 60 minutes instead" and Bob will update the file and re-validate EARS notation automatically.

6 Start Coding — Task by Task

Implement each task with automatic pre/post hooks and real-time status tracking

Tell Bob to Implement the First Task

In the Bob chat, send:

You

> Implement TASK-001: Create the User DB model and Alembic migration.

Bob will first fire the pre-task hook, then write the code, then fire the post-task hook:

MCP TOOL CALL — PRE-TASK HOOK

```
run_hook
hook_type: "pre_task"
feature_name: "user-authentication"
task_id: "TASK-001"
```

```
✅ PRE-TASK HOOK PASSED
— Spec files verified: requirements.md ✓, design.md ✓, tasks.md ✓
— TASK-001 status: todo → in_progress
```

✅ Bob will now write the `User` model in `backend/src/models/user.py` and the Alembic migration — exactly where `structure.md` says it should go.

Track Task Progress in Real Time

After Bob finishes the code, it automatically marks the task as complete and updates `tasks.md`:

TASKS.MD — USER-AUTHENTICATION

✅ Done	~~TASK-001: Create User DB model and Alembic migration~~
🔄 In Progress	TASK-002: Implement /api/v1/auth/register endpoint
⬜ Todo	TASK-003: Implement email verification service
⬜ Todo	TASK-004: Implement /api/v1/auth/login with JWT
⬜ Todo	TASK-005: Implement account lockout logic
⬜ Todo	TASK-006: Implement /api/v1/auth/reset-password flow
⬜ Todo	TASK-007: Write integration tests for all auth endpoints

⚡ Bob enforces task dependencies. If you ask it to implement TASK-004 before TASK-001 is done, it will refuse and remind you of the dependency chain defined in `tasks.md`.

Repeat for Every Feature

For each new feature, simply describe it to Bob in Spec Architect mode. The full cycle — **Requirements → Design → Tasks → Code** — runs automatically every time. Your steering files ensure every spec fits your project perfectly without you having to repeat yourself.

✅ Once IBM Bob access is available, the workflow is identical. The `.bob/` folder you already have in your project is all Bob needs to pick up where you left off.

🎉 Your Project Is Spec-Driven

Every feature now starts with a validated spec before a single line of code is written. IBM Bob enforces the workflow automatically on every request.

0
Lines of code written before the spec

3
Spec files generated per feature

6
MCP tools enforcing the workflow

1
Message needed to start a feature